

Breaking the Loop: Innovating Discovery on Spotify

CSC-475: Seminar in Computer Science
Spring 2024

William Foster

will.foster@furman.edu
Department of Computer Science
Furman University
will.foster@furman.edu

Abstract

While Spotify and other music streaming platforms excel at tailoring song recommendations to individual tastes through highly accurate algorithms, this precision often restricts users' musical exploration by promoting familiarity over discovery. This paper critiques the conventional feedback loop mechanisms in recommendation systems that hinder the exploration of new musical territories. An innovative algorithm is introduced, designed to break this cycle by enabling a broader, more adventurous musical journey for users wishing to diverge from their regular listening habits. Utilizing a dataset of approximately 90,000 unique songs from Spotify, supplemented by music metadata from MusicBrainz, the research develops and tests a new model that integrates user preferences with potential musical discoveries. The methodology employs data pre-processing, algorithm testing, and implementation, with significant modifications detailed to enhance understanding of the research process. The results demonstrate the effectiveness of the new algorithm, highlight the user responses from subsequent surveys, and assess the functionality of a purpose-built website to showcase the algorithm's capabilities. The study not only proposes a solution to the limitations of current music recommendation systems but also expands the scope of user interaction with music streaming platforms.

1 Introduction

Spotify and various music streaming platforms often face criticism for its seemingly uninspired song recommendation algorithms. Paradoxically, these algorithms are not flawed due to a lack of precision but rather their excessive accuracy. Spotify's system operates on a feedback loop mechanism, meticulously identifying a user's preferences to suggest similar tracks. This becomes an issue for those yearning to diverge from their habitual music selections, seeking to explore beyond their accustomed genres, eras, or tastes. Unfortunately, the algorithm's design does not cater to this desire for musical discovery, leaving users to embark on the search for new songs manually [1].

Exploring new music often proves to be a challenging endeavor, particularly for listeners attempting to diverge from their usual preferences. Despite their efforts to branch out, many users find themselves repeatedly drawn back to familiar tracks, caught in a perpetual cycle of repetitive song recommendations. Such challenges inspire this paper, and the research presented here introduces an algorithm designed to disrupt this loop, offering a fresh approach to music discovery. Further research extends to creating an API capable of analyzing user preferences to propose musical discoveries harmoniously with the developed algorithm.

The main dataset considered comes from Kaggle. The data includes 90,000 unique songs on Spotify and includes attributes like danceability, valence, and time signature, to name a few. The main collaborator, Maharshi Pandya, downloaded the main catalog from MusicBrainz, an open-source encyclopedia that collects music metadata. Subsequently, he queried each instance to match the

song scraped from the Spotify API, thus completing his data. The data was published in 2020 and is reportedly updated monthly [2].

The methods section provides a detailed description of the workflow pipeline used in the research. This includes the initial stages of data pre-processing, followed by extensive experimentation with various algorithms, and culminating in the implementation and finalization of the findings. Additionally, this section carefully outlines any significant shifts or pivots in the methodology, explaining the reasons and impacts of these changes on our overall research approach.

The results section presents a comprehensive overview of the findings, detailing the performance of the algorithm, the outcomes of the survey, and the functionality of the accompanying website. This section integrates all aspects of our research to provide a clear understanding of each component's effectiveness and interplay.

2 Related Work

In the digital music era, recommendation systems are pivotal in shaping user experience on platforms like Spotify. The examination presented delves deeply into the progress and obstacles associated with algorithms for music recommendations, scrutinizing the equilibrium among personalization, equity, variety, and user involvement. Its purpose is to frame the creation of an innovative algorithm intended to mirror listeners' genuine tastes more precisely, thereby enriching the continuous conversation within this domain.

The quest for fairness in algorithmic recommendations is a critical concern, as highlighted by Zehlike et al., which scrutinizes the integration of fairness into learning-to-rank (LtR) methods [3]. The paper elucidates various fairness-enhancing interventions, offering a comprehensive framework for understanding and mitigating bias in recommendation systems. This foundational perspective is crucial for acknowledging the ethical dimensions that underpin algorithmic decision-making processes. In parallel, the article by Anderson et al. examines Spotify's recommendation algorithms' impact on musical diversity, uncovering a tendency towards homogeneity in users' listening patterns. The research reveals how algorithmic curation can inadvertently narrow the spectrum of musical exposure, potentially stifling users' exploration of diverse genres and artists [4]. This finding underscores the importance of designing recommendation algorithms that promote a more comprehensive array of musical experiences, enhancing user satisfaction and engagement.

Diving into the technical mechanisms, data scientists at Technological University Dublin provide an exhaustive tutorial on k-Nearest Neighbour (k-NN) classifiers, a fundamental technique in recommendation systems [5]. The paper discusses the selection of similarity and distance metrics, the curse of dimensionality, and the strategies for improving computational efficiency and accuracy, such as dimension reduction. This exploration is pivotal for understanding how k-NN classifiers can be optimized for music recommendation, offering a balance between simplicity and effectiveness [5]. Advancements in deep learning for music recommendations are further explored in Maheshwari's article, which delves into user data and audio analysis to tailor song suggestions. The study highlights the superiority of deep learning models in capturing intricate patterns in user preferences and audio features, paving the way for highly personalized music recommendations [6]. Additionally, the article by Wu et al. surveys the application of Graph Neural Networks (GNNs) in recommender systems, emphasizing their adeptness at modeling complex user-item interactions [7]. This review introduces a new taxonomy for GNN-based models, addressing the challenges and future directions in leveraging GNNs for more nuanced recommendation algorithms.

The evaluation of recommender systems is addressed in the article titled, "Evaluating Recommender Systems: Survey and Framework", which proposes the Framework for Evaluating Recommender Systems (FEVR) [8]. This framework categorizes the diverse objectives, designs, and metrics involved in recommender system evaluation, advocating for a holistic approach that encompasses accuracy, user experience, and transparency [8]. User interaction with recommendation systems is further examined in "Controlling Spotify Recommendations: Effects of Personal Characteristics on Music Recommender User Interfaces", focusing on how personal characteristics influence users' preferences for visual control mechanisms in Spotify's recommendations [9]. The study's findings on the efficacy of radar charts over sliders in discovering new music provide valuable insights into designing more intuitive and personalized user interfaces for music recommendation systems.

The complex relationship between users and recommendation algorithms is explored in the study by Freeman et al., which investigates users' perceptions of and interactions with Spotify's recommendation features. The study illuminates how users anthropomorphize algorithms and employ various tactics to influence recommendation outcomes, reflecting the intricate socio-technical dynamics at play [10]. Dinnessen et al. offers a perspective from the music industry, highlighting concerns about fairness, diversity, and control in music streaming services. The paper calls for greater transparency and inclusivity in music recommendations, echoing the broader industry's apprehensions about the impact of algorithmic curation on music discovery and artist exposure [11].

Finally, the article by Hoi called "Music content personalized recommendation system based on a convolutional neural network," introduces a cutting-edge approach to music recommendations using convolutional neural networks (CNNs). The study showcases the CNN model's efficacy in dealing with the challenges posed by extensive music libraries, demonstrating its potential to significantly enhance the accuracy and personalization of music recommendations [12].

This examination covers ethical, technical, and user-focused aspects of music recommendation systems, underscoring the importance of creating algorithms that harmonize personalization with fairness and diversity. Insights from the sources discussed offer a thorough context for the algorithm being proposed, aiming to fill existing voids and propel forward the technology behind music recommendations. With a commitment to ethical principles, the utilization of sophisticated technical approaches, and a focus on user interaction, the endeavor seeks to enhance the digital musical journey for listeners.

3 Data

The dataset utilized in this study was specifically developed to enhance the construction of music recommendation systems, which tailor selections to user preferences and classify songs by audio features and genres. Compiled by Kaggle author Maharshi Pandya, it includes approximately 114,000 songs, with 89,741 unique tracks and 31,438 unique artists across 125 music genres. Pandya constructed this dataset by employing web-scraping techniques to extract data from the MusicBrainz music metadata site and the Spotify Web API, aligning songs with their corresponding metadata. While Pandya intends to maintain the dataset in its current state, he plans to expand it by integrating more comprehensive datasets in future research.

The dataset includes 21 diverse attributes, primarily sourced from MusicBrainz. These attributes encompass a range of data points such as track ID, artist name, album name, track title, popularity, duration (in milliseconds), explicit content marker, danceability, energy, musical key, loudness, mode, speechiness, acousticness, instrumentality, liveness, valence, tempo, time signature, and track genre. While certain attributes might appear arbitrary, their relevance and accuracy are not disputed in this context. Logistic regression was employed to identify and eliminate redundant or non-predictive attributes to manage the extensive range of attributes effectively.

Logistic regression is an analytical method frequently utilized in machine learning for binary classification tasks. It involves using a logistic function to estimate the probability that a given input belongs to a particular category, which, for example, is the likelihood of a song being performed by the band Coldplay. The statistical method computes coefficients for each attribute or feature under consideration, thereby quantifying the influence of each feature on the outcome's probability. The logistic model is mathematically denoted as $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$, where p signifies the probability of the song being by Coldplay, $\beta_0, \beta_1, \dots, \beta_n$ represent the coefficients, and $x_1, x_2, \dots, x_n$ symbolize the predictive features.

These coefficients are derived through a process known as maximum likelihood estimation, aiming to maximize the likelihood function to render the observed data as probable as possible given the model. In essence, a positive coefficient indicates that an increase in that feature's value is associated with a higher likelihood of the song being by Coldplay, and conversely, a negative coefficient implies a lower likelihood.

For practical implementation in Python, libraries such as Pandas and Scikit-learn offer efficient tools for data manipulation and model building, respectively. The dataset of songs, including every feature, was processed and loaded into a Pandas DataFrame. Following data pre-processing, which

Attribute	Definition
Acousticness	A confidence measure from 0.0 to 1.0 of whether the track is acoustic. 1.0 represents high confidence the track is acoustic.
Danceability	Danceability describes how suitable a track is for dancing based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity. A value of 0.0 is least danceable and 1.0 is most danceable.
Energy	Energy is a measure from 0.0 to 1.0 and represents a perceptual measure of intensity and activity. Typically, energetic tracks feel fast, loud, and noisy. For example, death metal has high energy, while a Bach prelude scores low on the scale.
Instrumentalness	Predicts whether a track contains no vocals. "Ooh" and "aah" sounds are treated as instrumental in this context. Rap or spoken word tracks are clearly "vocal". The closer the instrumentalness value is to 1.0, the greater likelihood the track contains no vocal content.
Genre	The genre in which the track belongs.
Speechiness	Speechiness detects the presence of spoken words in a track. The more exclusively speech-like the recording (e.g. talk show, audio book, poetry), the closer to 1.0 the attribute value.
Valence	A measure from 0.0 to 1.0 describing the musical positiveness conveyed by a track. Tracks with high valence sound more positive (e.g. happy, cheerful, euphoric), while tracks with low valence sound more negative (e.g. sad, depressed, angry.)

Table 1: Selected attributes and their definitions.

involved encoding categorical variables or imputing missing values, logistic regression could be performed.

The Scikit-learn library was used to fit the model to the dataset. The goal was to identify patterns specific to Coldplay’s music. The fitting procedure was as follows: the `LogisticRegression` function was initiated, and the model was trained with features stored in `X`, and the target variable, obtained by checking if Coldplay was present in the *artists* column of the `DataFrame`, converted to lowercase for consistency. The binary target variable represents whether a song is by Coldplay (1) or not (0). Once the model was fitted, the `.coef_` attribute of the logistic regression object reveals the influence (positive or negative) of each feature. By inspecting the `feature_importance`, one can determine which features have the most substantial impact on predicting Coldplay’s songs.

The features with the most prominent coefficients were picked by hand and earmarked for further analysis. The selected attributes, along with their descriptions, are presented in **Table 1**. Despite the seemingly subjective nature of this selection process, it represented the most pragmatic approach to distilling a subset of attributes for subsequent examination.

4 Methods

Following the initial data pre-processing steps, where information gain was utilized for attribute selection, the dataset underwent a thorough cleaning process. This included the removal of NA values and duplicate entries. It was observed that approximately 40 entries contained NA values. Furthermore, many songs appeared multiple times across original, best-of, and live albums. In instances where the metadata for these duplicates varied, the mean of each attribute was calculated to ensure consistency.

During the de-duplication phase, attention was paid to the distinction between different versions of songs that might share the same title but differ in their metadata due to their release on various album types. By the conclusion of this process, the dataset was refined to a collection of 73,609 unique songs, each characterized by meticulously averaged attributes where necessary. This approach not only streamlined the dataset but also enhanced the reliability of any subsequent analyses conducted on this data.

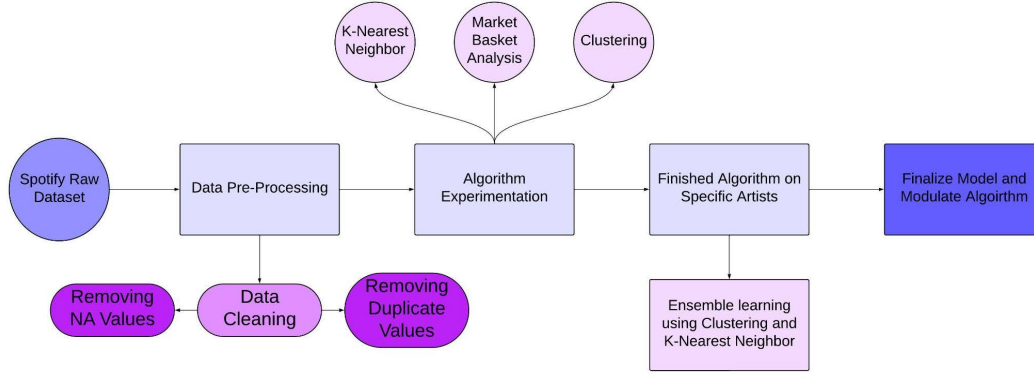


Figure 1: Algorithm pipeline

After the conclusion of the pre-processing stage, the process of algorithm experimentation began. Market basket analysis was the first method used for experimentation, but produced no tangible results. Market basket analysis is a data mining technique used to identify relationships between items in large datasets commonly used in retail to analyze customer purchasing patterns. This method involves the discovery of items that frequently co-occur in transactions through the use of association rules, which measure the likelihood of items being purchased together. These rules are quantified by metrics such as support, confidence, and lift. In the context of Spotify, market basket analysis can be adapted to analyze listening habits instead of purchasing behaviors. When applied to music streaming, the technique identifies patterns in the songs that users listen to together within a certain time frame. For instance, if many users frequently listen to Song A and Song B consecutively or within the same session, Spotify might suggest Song B to a listener of Song A, anticipating that there is a high probability they will enjoy it based on the observed pattern. Unfortunately, without user data and support and confidence measures, it was not a viable option for analysis.

The subsequent experiments utilized an instance-based learning method known as the k-nearest neighbor (k-NN) algorithm, as outlined in Cunningham’s article [5]. This method is straightforward but potent and suitable for classification and regression tasks. The k-NN algorithm works on a fundamental principle: it determines the label of a data point by examining the ‘k’ closest labeled data points to it, then either taking a majority vote (for classification) or computing the average (for regression) of their labels.

Initially, the number of neighbors, ‘k’, is selected. When a new, unlabeled data point requires classification, the algorithm computes the distance—commonly using the Euclidean metric—from this point to all others in the training dataset. It then identifies the ‘k’ nearest neighbors or the ‘k’ training points most proximate to the query point. The prediction is based on the most prevalent class among these neighbors or the average outcome in regression cases. Notably, k-NN is a non-parametric method; it does not assume any underlying distribution of the data, which adds to its flexibility and makes it particularly sensitive to the local data structure. This general behavior of the algorithm is depicted in **Figure 2**.

Challenges arose due to the absence of a predetermined clustering method, leading to mixed results when testing with a neighborhood size of 10. Additionally, the lack of many popular songs in the dataset complicated the identification of appropriate test instances. These issues suggested a potential challenge in the survey stage, prompting the need for a strategic pivot.

After some deliberation, the decision was made to start with a broader approach: making recommendations based on users’ favorite artists. This method provided a more structured foundation for the initial phase of the algorithm—clustering. Clustering is an unsupervised learning technique in machine learning and data analysis that groups objects such that those within the same cluster are more similar to each other than to those in other clusters. This technique organizes data into clusters based on shared characteristics. Common clustering algorithms include k-means, hierarchical clustering, and DBSCAN. Typically, the algorithm determines or discovers the required number of clusters from the data. It then assigns each data point to a cluster based on a specific criterion. For example, k-means minimizes the distance between data points and their cluster’s center, whereas

The diagram shows the equation for the objective function J in k-means clustering. The equation is $J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$. Annotations include:

- An arrow from "number of clusters" pointing to the upper limit k of the first summation.
- An arrow from "number of cases" pointing to the upper limit n of the second summation.
- An arrow from "case i " pointing to the index i in $x_i^{(j)}$.
- An arrow from "centroid for cluster j " pointing to c_j .
- An arrow from "objective function" pointing to J .
- A bracket under the term $\|x_i^{(j)} - c_j\|^2$ is labeled "Distance function".

Figure 2: General equation used for calculating nearest neighbor.

hierarchical clustering builds a tree of clusters. The outcome is a collection of clusters that reveal the inherent groupings and patterns within the data, thus aiding in structured data analysis and decision-making.

By employing clustering techniques, the algorithm was able to effectively group songs from each artist into three distinct clusters. This was achieved by normalizing the metadata of each song and utilizing the k-means clustering algorithm to assign songs to their respective clusters. Within each cluster, the top song was identified to represent the most prominent attribute of that group. This representative song was then used as a benchmark to find the closest matching song within the same cluster, facilitating a more nuanced and targeted approach to song recommendation based on distinct musical characteristics. An example of clustering songs based on normalized attributes can be found in **Figure 3**.

With a more structured clustering method now in place, it was possible to revisit the k-nearest neighbor algorithm for further experimentation. By clustering the remainder of the dataset - not just by the artist but by selected characteristics - the neighborhoods formed were more closely representative of the top songs from each cluster. This approach yielded significantly improved results compared to the preliminary experiments. The specific outputs of the algorithm can be found in the results section.

The final algorithm exemplifies a machine learning strategy known as ensemble learning. This paradigm involves training multiple models, often called "weak learners," to address the same problem and then combining their outputs to enhance overall performance. This approach typically surpasses the effectiveness of using a single model by capitalizing on the strengths and counteracting the weaknesses of the individual models.

For example, employing ensemble techniques with k-nearest neighbor and clustering algorithms can significantly boost prediction accuracy and reliability. Initially, clustering segregates the data into distinct groups based on similarities among the data points. Each group, or cluster, then serves as a separate subset for training individual k-NN models. Subsequently, the predictions from these multiple k-NN models—each trained on a different cluster—are aggregated. This aggregation is generally done by voting in classification tasks or averaging for regression, culminating in a robust final prediction.

This method effectively leverages the localized learning capabilities of k-NN within the homogeneously formed clusters, thereby enhancing both the robustness and the accuracy of predictive modeling across diverse or extensive datasets, as demonstrated by the algorithm in question. A full layout of the algorithm can be seen in **Algorithm 1** below.

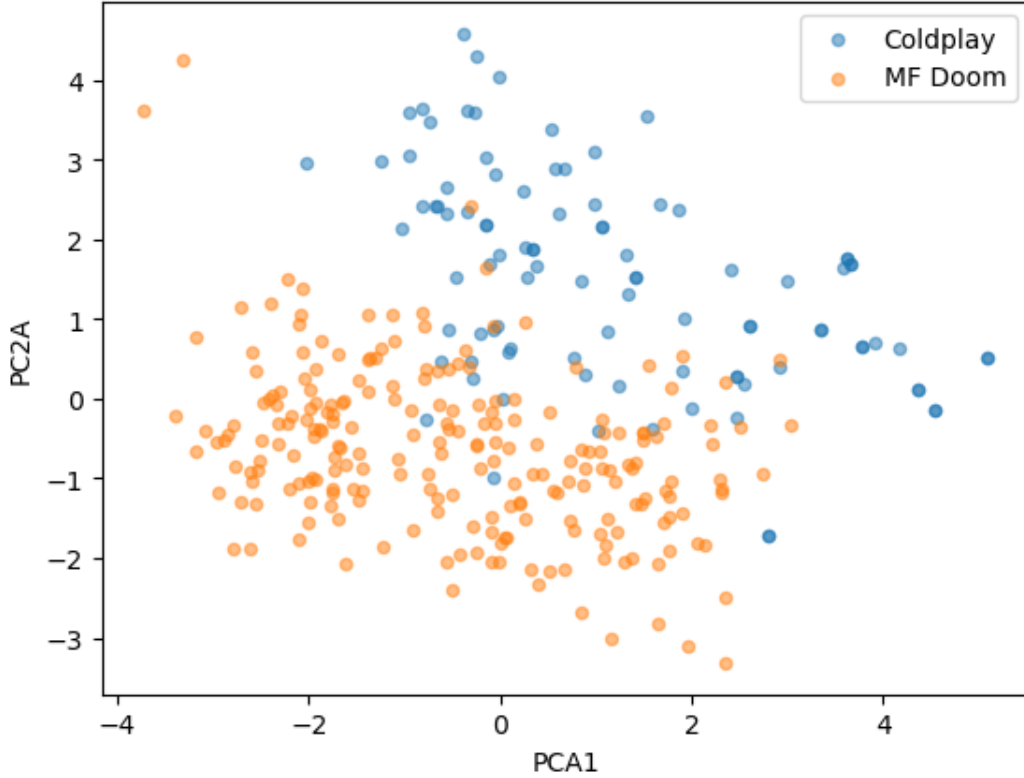


Figure 3: Example of song clustering by attributes. Artists Coldplay and MF DOOM were chosen to show attribute disparity.

Algorithm 1 Ensemble Learning with k-NN and Clustering

Result: Final prediction from ensemble of k-NN models trained on clustered data

Step 1: Clustering Initialize number of clusters K **for each data point** x_i **in the dataset do**

 | Assign x_i to a cluster C_k based on similarity measures

end

Step 2: Train k-NN models **for each cluster** C_k **do**

 | Train a k-NN model M_k using data points in C_k

end

Step 3: Aggregation for Prediction **for each new instance** x **do**

for each model M_k **do**

 | Compute prediction p_k for x using M_k

end

 Aggregate predictions $\{p_1, p_2, \dots, p_K\}$ (by voting or averaging) Output the final aggregated prediction

end

Each step described above was performed in a Jupyter Notebook utilizing pandas. Libraries like `sklearn.cluster` (KMeans) and `cdist` (`scipy.spatial.distance`) were used for clustering and k-NN, respectively. The code outlined by the methodology can be found at <https://github.com/wfoster01/csc475spotify>.

5 Results

5.1 Algorithm Results and Testing

As previously noted, the algorithm is executed within a Jupyter Notebook. Upon running, it prompts the user to enter three of their favorite artists. The prompt requests an alternative artist if an entered artist is absent in the dataset or lacks unique songs. After selecting valid artists, the algorithm proceeds to cluster the songs of these artists based on various attributes, subsequently outputting the top songs from each cluster. The clustering results are displayed in **Table 2**. Additional code is then utilized to select the most representative song from each cluster, enhancing the personalization of the recommendations.

Track Name	Cluster
Hymn for the Weekend	2
Christmas Lights	1
Yellow	2
Yellow	2
Green Eyes	0
Christmas Lights	1
The Scientist	0
Christmas Lights	1
Higher Power	2
Christmas Lights	1
Everyday Life	0

Table 2: Example of cluster outputs.

After selecting the top three songs from each artist, the algorithm employs the k-nearest neighbor method to identify the closest songs to these top tracks, focusing on songs by different artists. The output generated by the code includes the name of the original artist, one of their top songs, and the closest matching song, along with its respective artist’s name. An example of the output can be seen in **Table 3**. A website that takes in the user’s artist and outputs songs can be found here: <http://fahadsultan.com/csc475spotify/>.

Non-Coldplay song closest to 'Green Eyes' by Coldplay: 'Infinity' by Jaymes Young
Non-Coldplay song closest to 'Christmas Lights' by Coldplay: 'The Universal' by Blur
Non-Coldplay song closest to 'Hymn for the Weekend' by Coldplay: 'Black Balloon' by The Goo Goo Dolls

Table 3: Example of final song output.

The algorithm was evaluated using five college students, each expressing a constant desire for new music. Initially, each participant was asked to identify their three favorite artists, and these selections were processed through the algorithm. A new Spotify account was created to eliminate any influence from user data on personal accounts and ensure an unbiased comparison with Spotify.

Once participants input their preferences into the algorithm, they were asked to listen to three songs—one from each selected artist. Subsequently, they were queried about their familiarity with the songs and artists and whether they enjoyed the songs. For the comparative analysis with Spotify, participants searched for each artist on Spotify, navigated to the artist’s radio, and noted the first song that was not by the artist themselves.

5.2 Survey Results

The survey results, seen in **Figure 4**, revealed that the algorithm effectively recommended new music to participants, with only 2 out of 15 songs being previously recognized. Conversely, Spotify’s

Experimental Algorithm	P1	P2	P3	P4	P5	Total
How many songs have you heard before?	0/3	0/3	1/3	0/3	0/3	2/15
How many artists have you heard of before?	2/3	1/3	1/3	1/3	1/3	8/15
How many songs do you like?	2/3	3/3	3/3	3/3	2/3	13/15
Spotify's Algorithm	P1	P2	P3	P4	P5	Total
How many songs have you heard before?	2/3	2/3	3/3	3/3	3/3	13/15
How many artists have you heard of before?	2/3	3/3	3/3	3/3	3/3	14/15
How many songs do you like?	2/3	3/3	3/3	3/3	3/3	13/15

Table 4: Comparison of Experimental and Spotify's Algorithms.

algorithm predominantly offered familiar tracks, including Van Morrison's "Brown Eyed Girl" to three individuals.

While limited in scope, the survey provided a basic insight into the effectiveness of Spotify's recommendation algorithm. It appears that the algorithm primarily suggests songs based on their popularity and overall listening frequency rather than introducing new music. Although most participants appreciated the songs recommended by Spotify, they noted these were tracks they had been familiar with for years, often songs they had already encountered through their own accounts.

In contrast, the experimental algorithm yielded an equal number of liked songs but introduced tracks that participants were unfamiliar with. This algorithm was particularly effective in uncovering "deep tracks"—lesser-known songs by popular artists overshadowed by their hits. Participants emphasized that without this experimental algorithm, they would likely never have discovered these hidden gems.

6 Future Work

The dataset currently used, which consists of approximately 80,000 unique songs, presents a significant limitation in our ability to provide a wide range of musical diversity. This restriction is particularly impactful as it constrains the array of artists and genres accessible to users, limiting the effectiveness of our recommendation system in fostering true musical discovery and exploration. The finite size of the dataset means that users may not find lesser-known artists or tracks, potentially reinforcing the very loops of familiar content our project aims to break.

To address this limitation, an expansion of our dataset is crucial. Integrating the Spotify API could be a pivotal improvement. The Spotify API allows dynamic access to an extensive library of music, including millions of songs, playlists, and artist profiles. By leveraging this API, we can enrich our dataset in real time, tailoring recommendations more closely to individual user preferences. This method not only expands the diversity of our recommendations but also enhances the personalization aspect of our service, which is vital for engaging users in a meaningful exploration of music.

Incorporating user interaction data through the Spotify API can further refine our recommendation algorithms. By analyzing data on what users play, save, or skip, we can gain insights into their musical tastes beyond the static attributes currently utilized. This dynamic data integration would allow our algorithm to adapt continually and evolve in response to user feedback, potentially introducing users to a broader spectrum of music while still aligning with their tastes.

The challenge, however, lies in implementing these enhancements. Integrating the Spotify API requires careful consideration of data privacy and user consent. Furthermore, the increase in dataset size and complexity may demand more robust computational resources and refined algorithms capable of processing and making sense of the vast amount of data efficiently.

In conclusion, by expanding our dataset through the Spotify API and leveraging real-time user data, we can significantly enhance the scope and accuracy of our music recommendation system. This approach not only addresses the limitations posed by the current dataset size but also sets the stage for a more adaptive and user-focused recommendation model, promoting a richer and more diverse musical discovery experience.

References

- [1] Victoria Turk. How to break out of your spotify feedback loop and find new music, Jan 2021.
- [2] Maharshi Pandya. Spotify tracks dataset, 2022.
- [3] Meike Zehlike, Ke Yang, and Julia Stoyanovich. Fairness in ranking, part ii: Learning-to-rank and recommender systems. *ACM Comput. Surv.*, 55(6), Dec 2022.
- [4] Ashton Anderson, Lucas Maystre, Ian Anderson, Rishabh Mehrotra, and Mounia Lalmas. Algorithmic effects on the diversity of consumption on spotify. In *Proceedings of The Web Conference 2020, WWW '20*, page 2155–2165, New York, NY, USA, 2020. Association for Computing Machinery.
- [5] Pádraig Cunningham and Sarah Jane Delany. k-nearest neighbour classifiers - a tutorial. *ACM Comput. Surv.*, 54(6), Jul 2021.
- [6] Chhavi Maheshwari. Music recommendation on spotify using deep learning, 2023.
- [7] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. Graph neural networks in recommender systems: A survey. *ACM Comput. Surv.*, 55(5), Dec 2022.
- [8] Eva Zangerle and Christine Bauer. Evaluating recommender systems: Survey and framework. *ACM Comput. Surv.*, 55(8), Dec 2022.
- [9] Martijn Millecamp, Nyi Nyi Htun, Yucheng Jin, and Katrien Verbert. Controlling spotify recommendations: Effects of personal characteristics on music recommender user interfaces. In *Proceedings of the 26th Conference on User Modeling, Adaptation and Personalization, UMAP '18*, page 101–109, New York, NY, USA, 2018. Association for Computing Machinery.
- [10] Sophie Freeman, Martin Gibbs, and Bjørn Nansen. ‘don’t mess with my algorithm’: Exploring the relationship between listeners and automated curation and recommendation on music streaming services. *First Monday*, 27(1), Jan. 2022.
- [11] Karlijn Dinnissen, Isabella Saccardi, Marloes Vredenburg, and Christine Bauer. Looking at the facets: Exploring music industry professionals’ perspectives on music streaming services and recommendations. In *Proceedings of the 2nd International Conference of the ACM Greek SIGCHI Chapter, CHIGREECE '23*, New York, NY, USA, 2023. Association for Computing Machinery.
- [12] Rui Hoi. Music content personalized recommendation system based on a convolutional neural network. 28(2):1785–1802, Feb 2023.